

LISTS

Introduction to Lists

A list is an ordered, mutable collection used to store multiple values in a single variable.

Lists are created using: `[]` or `list()`

Example:

Python

```
numbers = [10, 20, 30]
names = ["Ravi", "Teja", "Ankit"]
mixed = [10, "Python", 5.5, True]
```

LIST PROPERTIES

Property	Description
Ordered	Elements maintain insertion order
Mutable	Elements can be modified
Indexed	Elements can be accessed using index
Iterable	Can iterate using loops
Allows Duplicates	Duplicate values are allowed
Dynamic Size	Size can grow or shrink
Heterogeneous	Can store different datatypes

LIST OPERATIONS

1. Concatenation (+)

Used to join lists.

Python

```
a = [1, 2]
b = [3, 4]

print(a + b)
[1, 2, 3, 4]
```

2. Repetition (*)

Used to repeat list elements.

Example:

Python

```
print([1, 2] * 3)

[1, 2, 1, 2, 1, 2]
```

3. Indexing

Used to access elements using index positions.

Example:

Python

```
data = [10, 20, 30, 40]
print(data[0])
print(data[-1])

10
40
```

4. Slicing

Used to extract part of a list.

Syntax: `list[start:end:step]`

Python

```
data = [10, 20, 30, 40, 50]
print(data[1:4])
print(data[::-1])
```

```
[20, 30, 40]
```

```
[50, 40, 30, 20, 10]
```

5. Membership Operators

Checks whether an element exists.

Python

```
data = [10, 20, 30]

print(20 in data)
print(100 not in data)
```

```
True
```

```
True
```

BUILT-IN LIST FUNCTIONS

Function	Description	Example
<code>len()</code>	Returns list length	<code>len([1,2,3]) → 3</code>
<code>max()</code>	Largest element	<code>max([10,20,5]) → 20</code>
<code>min()</code>	Smallest element	<code>min([10,20,5]) → 5</code>
<code>sum()</code>	Sum of elements	<code>sum([1,2,3]) → 6</code>
<code>sorted()</code>	Returns sorted list	<code>sorted([3,1,2])</code>
<code>list()</code>	Converts iterable to list	<code>list("abc")</code>

LIST METHODS

List methods are predefined functions used to perform operations on lists.

1. ADDING ELEMENT METHODS

Method	Description	Example
append()	Adds single element at end	lst.append(10)
extend()	Adds multiple elements	lst.extend([1,2])
insert()	Inserts element at position	lst.insert(1,100)

2. REMOVING ELEMENT METHODS

Method	Description	Example
remove()	Removes first occurrence	lst.remove(10)
pop()	Removes element using index	lst.pop()
clear()	Removes all elements	lst.clear()
del	Deletes list or element	del lst[0]

3. SEARCH METHODS

Method	Description	Example
index()	Returns element index	lst.index(20)
count()	Counts occurrences	lst.count(10)

4. SORTING & REVERSING METHODS

Method	Description	Example
sort()	Sorts original list	lst.sort()
reverse()	Reverses list	lst.reverse()
sorted()	Returns sorted new list	sorted(lst)

5. COPYING METHODS

Method	Description	Example
copy()	Creates shallow copy	lst.copy()

NESTED LISTS

A list inside another list is called a nested list.

Example:

Python

```
data = [[1, 2], [3, 4]]
```

```
print(data[0])
```

```
print(data[1][1])
```

```
[1, 2]
```

```
4
```

TUPLES

Introduction to Tuples

A tuple is an ordered, immutable collection used to store multiple values in a single variable.

Tuples are created using parentheses () or tuple()

Python

```
numbers = (10, 20, 30)
names = ("Ravi", "Teja", "Ankit")
mixed = (10, "Python", 5.5, True)
```

TUPLE PROPERTIES

Property	Description
Ordered	Elements maintain insertion order
Immutable	Elements cannot be added, removed, or modified directly
Iterable	Can iterate through elements using loops
Allows Duplicates	Duplicate values are allowed
Heterogeneous	Can store different datatypes
Supports Nested Objects	Can contain tuples, lists, sets, dictionaries, etc.
Mutable Objects Can Change	If a tuple contains mutable objects (like lists), those objects can be modified
Faster Than Lists	Generally consumes less memory and provides faster access than lists

CREATING TUPLES

Empty Tuple

```
Python  
t = ()
```

Single Element Tuple

```
Python  
t = (10,)
```

Note:

```
Python  
t = (10)
```

is treated as an integer, not a tuple.

TUPLE OPERATIONS

1. Concatenation (+)

Used to join tuples.

```
Python  
a = (1, 2)  
b = (3, 4)  
  
print(a + b)      # (1, 2, 3, 4)
```

2. Repetition (*)

Used to repeat tuple elements.

```
Python  
print((1, 2) * 3)      # (1, 2, 1, 2, 1, 2)
```

3. Indexing

Used to access elements using index positions.

Python

```
data = (10, 20, 30, 40)

print(data[0])      # 10
print(data[-1])     # 40
```

4. Slicing

Used to extract part of a tuple.

Syntax:

Python

```
tuple[start:end:step]
```

Example:

Python

```
data = (10, 20, 30, 40, 50)

print(data[1:4])      # (20, 30, 40)
print(data[::-1])     # (50, 40, 30, 20, 10)
```

5. Membership Operators

Checks whether an element exists.

Python

```
data = (10, 20, 30)

print(20 in data)      # True
print(100 not in data) # True
```


BUILT-IN TUPLE FUNCTIONS

Function	Description	Example
len()	Returns tuple length	len((1,2,3)) → 3
max()	Returns largest element	max((10,20,5)) → 20
min()	Returns smallest element	min((10,20,5)) → 5
sum()	Returns sum of numeric elements	sum((1,2,3)) → 6
sorted()	Returns sorted list	sorted((3,1,2)) → [1,2,3]
tuple()	Converts iterable to tuple	tuple("abc") → ('a','b','c')
any()	Returns True if at least one value is True	any((0,0,1)) → True
all()	Returns True if all values are True	all((1,2,3)) → True

TUPLE METHODS

Since tuples are immutable, they provide only two built-in methods.

Method	Description	Example
count()	Counts occurrences of an element	(1,2,2,3).count(2) → 2
index()	Returns first occurrence index	(10,20,30).index(20) → 1

TUPLE PACKING

Storing multiple values into a tuple automatically.

Python

```
data = 10, 20, 30
```

```
print(data)      # (10, 20, 30)
```

TUPLE UNPACKING

Extracting tuple elements into variables.

Python

```
data = (10, 20, 30)

a, b, c = data

print(a)      # 10
print(b)      # 20
print(c)      # 30
```

NESTED TUPLES

A tuple inside another tuple is called a nested tuple.

Python

```
data = ((1, 2), (3, 4))

print(data[0])      # (1, 2)
print(data[1][1])   # 4
```

IMMUTABILITY OF TUPLES

Tuple elements cannot be modified.

Python

```
data = (10, 20, 30)

data[0] = 100

# TypeError
```

MUTABLE OBJECTS INSIDE TUPLES

Although tuples are immutable, mutable objects stored inside them can still be modified.

Python

```
data = (10, [20, 30], 40)

data[1].append(50)

print(data)

# (10, [20, 30, 50], 40)
```

Explanation:

Python

```
data[0] = 100
```

Not Allowed

Python

```
data[1].append(50)
```

Allowed

Because the tuple itself is immutable, but the list inside it is mutable.

WHY USE TUPLES?

- Data should not be modified.
- Better memory efficiency.
- Faster than lists.
- Useful for fixed collections of values.
- Commonly used for coordinates, dimensions, and database records.
- Supports packing and unpacking.

Example:

Python

```
location = (17.3850, 78.4867)
dimensions = (10, 20, 5)
```

LIST vs TUPLE

Feature	List	Tuple
Syntax	[]	()
Example	[10, 20, 30]	(10, 20, 30)
Mutability	Mutable	Immutable
Add Elements	Allowed	Not Allowed
Remove Elements	Allowed	Not Allowed
Modify Elements	Allowed	Not Allowed
Ordered	Yes	Yes
Indexed	Yes	Yes
Iterable	Yes	Yes
Allows Duplicates	Yes	Yes
Heterogeneous Data	Yes	Yes
Nested Structures	Yes	Yes
Dynamic Size	Yes	No (fixed after creation)
Memory Usage	More Memory	Less Memory
Performance	Slightly Slower	Faster
Built-in Methods	Many Methods	Only 2 Methods
Sorting	sort() available	No sort() method
Copying	copy() available	No copy() method
Suitable For	Frequently changing data	Fixed data that should not change
Hashable	No	Yes (if all elements are immutable)
Dictionary Key	Cannot be used	Can be used (if hashable)